Java Bank Application Assignment

1. Download the **JavaBankApplication** project template from the course pages and use it as your starting point. You find the files **BankProgram**.java and **Account**.java in the project.

2. The bank application is described on pages 1-2 of this assignment paper.

3. NB! **Instruction on how to proceed** in writing the code starts on page 3.

In this assignment, you write an overly simplified 'bank application'. The functions of the application are the following:

- Open a new account
- Deposit money to an account
- Withdraw money from an account
- Delete an account
- List accounts.

Some business rules

- A bank account has the following two attributes: account number and balance.
- Account numbers are unique. That is, duplicate account numbers are not allowed.
- Negative account balances are not allowed.
- The balance of a new account is always zero.
- The balance of an account can be changed only by depositing money to the account or withdrawing money from the account.

TECHNICAL REQUIREMENTS

You will complete the two classes that are shown in the diagram below.

BankProgram
- accountList : ArrayList<Account>
+ main(String[] args) : void - addAccount() : void - deleteAccount() : void - depositMoney() : void - findAccount(String accountNumber): Account - listAccounts() : void - withdrawMoney() : void

Account
- accountNumber : String - balance : double
+ Account(accountNumber : String) + deposit(double amount) : void + getAccountNumber : String + getBalance() : double + withdraw(double amount) : boolean

You see the required fields, constructors, methods, method parameters and return types in the diagram above. In the diagram, '-' means private and '+' means public.

The Bank program class

A List will be used for saving account data during the program execution. The user is allowed to create any number of accounts. The account data is not saved permanently anywhere.

1. An empty list of Accounts is already created in the class BankProgram.

2. The **main** method of the BankProgram class is already completed. It contains the code for displaying the program menu shown below. **Do NOT modify the *main* method at all!**

0 = Quit | 1 = List accounts | 2 = Add an account | 3 = Deposit money | 4 = Withdraw money | 5 = Delete an account

Enter `your choice:`

3. The methods that you should complete are listed in the diagram on page 1. On page 3 you see examples of user interaction for each function of the bank application.

NB! The method **findAccount** searches for an Account object from the account list by given account number.

If the account is found, the method *returns a reference to the Account object*. Otherwise, the method *returns null*.

You should use the method **findAccount** to find the desired account in the following situations:
- Depositing money to the desired account
- Withdrawing money from the desired account
- Deleting the desired account
- When adding a new account to check for uniqueness of the desired account number.

That is, you first only once write the method that searches for an account and then use (call) the method in several different situations.

The Account class

You should complete the Account class exactly as shown in the diagram on page 1. In the diagram you see what fields, constructors and methods you should write in the Account class.

DO NOT add any other fields, constructors or methods to the Account class.

The methods of the class Account do NOT display anything or input anything directly from the user.

NB! The method **withdrawMoney** changes the balance and returns **true** if the available balance of the account is high enough to make the withdrawal, otherwise it returns **false**.

HOW TO PROCEED IN THIS ASSIGNMENT?

PHASE 1

Please write and test the program step by step as follows:

1. Write the required **fields**, **constructor** and **get-methods** of the Account class.
2. Write and test the methods **addAccount** and **listAccounts** of the BankProgram class.
3. Write the method **findAccount** of the BankProgram class.
4. Write the method **deposit** of the Account class.
5. Write and test the method **depositMoney** of the BankProgram class.
6. etc. etc.

Do **NOT** prepare yet for invalid input data, duplicate accounts, non-existent accounts etc. You do this later.

See the descriptions of the interaction between the user and the program below. The user input is highlighted in yellow.

Choice	Interaction	Comments
1	***** Account list ***** Number: 1231 | Balance: 100.25 euros Number: 323 | Balance: 8987.68 euros	All accounts are listed. The balance is always shown with two decimals.
2	***** Add an account ***** Enter account number: 2345 Account created successfully!	Balance of a new account is always zero.
3	***** Deposit money to an account ***** Enter account number: 2345 Enter the amount to be deposited: 125.25 Deposit completed successfully!	The balance of the account is increased by the given amount.
4	***** Withdraw money from an account ***** Enter account number: 2345 Enter the amount to be withdrawn: 15.00 Withdrawal completed successfully!	If the available balance of the account is high enough, then the balance is decreased by the given amount
4	***** Withdraw money from an account ***** Enter account number: 2345 Enter the amount to be withdrawn: 99999999 Withdrawal not completed. Available balance is too low.	If the available balance is too low for the withdrawal, then the balance is not changed.
5	***** Delete an account ***** Enter account number: 323 Account deleted successfully!	The account is deleted

PHASE 2

Please continue writing and testing the program step by step.

NOW you add the code for handling cases of some invalid input data, duplicate accounts, and non-existent accounts.

See the descriptions of the interaction between the user and the program below. The user input is highlighted in yellow.

Choice	Example	Comments
2	***** Add an account ***** Enter account number: 2345 Account not created. Account 2345 exists already!	Duplicate account numbers should not be allowed.
3	***** Deposit money to an account ***** Enter account number: 9999 Account 9999 does not exist!	It should not be possible to deposit money to a non-existent account.
3	***** Deposit money to an account ***** Enter account number: 2345 Enter the amount to be deposited: -123.25 Cannot deposit a negative amount!	It should not be possible to deposit a negative amount.
4	***** Withdraw money from an account ***** Enter account number: 9999 Account 9999 does not exist!	It is not possible to withdraw money from a non-existent account.
4	***** Withdraw money from an account ***** Enter account number: 2345 Enter the amount to be withdrawn: -123.25 Cannot withdraw a negative amount!	It should not be possible to withdraw a negative amount.
5	***** Delete an account ***** Enter account number: 9999 Nothing deleted. Account 9999 does not exist!	It is not possible to delete non-existent account.

IMPORTANT! (What the program should do after it has given a message to the user...)

This time, we simplify the program so that the program does NOT ask something again after it rejects the chosen operation. The program just shows the menu in the normal way after it has displayed a message to the user.